

Labo 19

opleidingsonderdeel **Web Development 1**

studiegebied **Handelswetenschappen, bedrijfskunde**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Sander Vandeputte

docent: **Pim Debaere**

academiejaar **2025-2026**

DEMO timers: setInterval

[illegible]

Stop

Deze code start bij het laden van de pagina een `setInterval` dat elke seconde automatisch het woord “tick” toevoegt aan het output-element. De timer blijft doorlopen tot de gebruiker op de knop “Stop” klikt. Bij het klikken wordt het interval gestopt met `clearInterval`, waardoor er geen nieuwe ticks meer verschijnen.

HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo</title>
</head>

<body>
  <p id="output"></p>
  <input id="btnStop" type="button" value="Stop">
  <script src="../scripts/demo_1_setinterval.js"></script>
</body>
</html>
```

Javascript

```
let timerId=0;

const initialize = () => {
    let btnStop=document.getElementById("btnStop");
    btnStop.addEventListener("click", stopTimer);
    timerId=setInterval(timerTick, 1000);
}

const timerTick = () => {
    let output=document.getElementById("output");
    output.innerHTML+=" tick";
}

const stopTimer = () => {
    clearInterval(timerId);
}

window.addEventListener("load", initialize);
```

DEMO timers: setTimeout eenmalig

tick tick tick

Opnieuw

Deze code start bij het laden van de pagina éénmalig een timer die na één seconde het woord “tick” toevoegt aan het output-element. Wanneer de gebruiker op de knop “Opnieuw” klikt, wordt opnieuw een eenmalige timer van één seconde gestart die nog een “tick” toevoegt. Er loopt dus geen doorlopende timer: elke tick verschijnt enkel na een afzonderlijke timeout.

HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo</title>
</head>

<body>
  <p id="output"></p>
  <input id="btnOpnieuw" type="button" value="Opnieuw">
  <script src="../../scripts/demo_2_settimeout_eenmalig.js"></script>
</body>
</html>
```

Javascript

```
const initialize = () => {
  let btnOpnieuw=document.getElementById("btnOpnieuw");
  btnOpnieuw.addEventListener("click", herstartTimer);
  setTimeout(timerTick, 1000);
}

const timerTick = () => {
  let output=document.getElementById("output");
  output.innerHTML+=" tick";
}

const herstartTimer = () => {
  setTimeout(timerTick, 1000);
}

window.addEventListener("load", initialize);
```

DEMO timers: setTimeout herhaald

tick tick tick tick tick tick tick tick tick tick tick tick tick tick tick tick tick tick tick

Stop

Deze code gebruikt setTimeout om een herhalende timer te maken. Bij het laden van de pagina wordt na één seconde een eerste “tick” toegevoegd aan het output-element. In timerTick() wordt telkens opnieuw een nieuwe timeout van één seconde gestart, waardoor de ticks blijven verschijnen. Wanneer de gebruiker op de knop “Stop” klikt, wordt de lopende timeout gestopt met clearTimeout, waardoor de herhaling stopt.

HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo</title>
</head>

<body>
  <p id="output"></p>
  <input id="btnStop" type="button" value="Stop">
  <script src="../scripts/demo_3_settimeout_herhaald.js"></script>
</body>
</html>
```

JavaScript

```
let timerId=0;

const initialize = () => {
  let btnStop=document.getElementById("btnStop");
  btnStop.addEventListener("click", stopTimer);
  timerId=setTimeout(timerTick, 1000);
}

const timerTick = () => {
  let output=document.getElementById("output");
  output.innerHTML+=" tick";
  timerId=setTimeout(timerTick, 1000);
}

const stopTimer = () => {
  clearTimeout(timerId);
}

window.addEventListener("load", initialize);
```

willekeurige getallen

<pre>let getal = Math.random();</pre>	Random getal tussen 0 en 1. 1 is niet inbegrepen.
<pre>let getal = Math.random() * 15;</pre>	Random getal tussen 0 en 15. 15 is niet inbegrepen.
<pre>let getal = 10 + Math.random()*15;</pre>	Random getal tussen 15 en 25. 25 niet inbegrepen.
<pre>let getal = Math.floor(Math.random() * 101);</pre>	Random getal tussen 0 en 101. 101 niet inbegrepen. Door floor worden een geheel getal weergegeven.

Gehele getallen bekomen

<pre>Math.floor(7.67)</pre>	Dit geeft 7 weer.
<pre>Math.floor(-7.34)</pre>	Dit geeft -8 weer.
<pre>Math.ceil(7.34)</pre>	Dit geeft 8 weer.
<pre>Math.ceil(-7.67)</pre>	Dit geeft -7 weer.
<pre>Math.round(7.34)</pre>	Dit geeft 7 weer.
<pre>Math.round(7.67)</pre>	Dit geeft 8 weer.
<pre>Math.round(7.5)</pre>	Dit geeft 8 weer.
<pre>Math.round(-7.5)</pre>	Dit geeft -7 weer.

DEMO sprites

Klik op het speelveld om de smiley te verplaatsen.



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo</title>
  <link rel="stylesheet" href="../styles/demo_sprites.css">
</head>

<body>
  <div id="speelveld" class="unselectable">
    Klik op het speelveld om de smiley te verplaatsen.
    
  </div>
  <Script src="../scripts/demo_sprites.js"></script>
</body>
```

CSS

```
html, body {
  margin:0px;
}

#speelveld {
  position:relative;
  border:solid 1px black;
  box-sizing:border-box; /* anders krijgen we scrollbars omdat het
element door de border net te groot wordt */
}

.sprite {
  position:absolute;
}

.unselectable {
```

```
-webkit-user-select: none;
-moz-user-select: none;
user-select: none;
}
```

Javascript

```
const setup = () => {
  window.addEventListener("resize", updateSize);

  // al 1 keer op voorhand oproepen, voor het geval
  // dat de gebruiker nooit manueel het browservenster
  // groter of kleiner maakt
  updateSize();

  let speelveld=document.getElementById("speelveld");
  speelveld.addEventListener("click", moveSprite);
}

const updateSize = () => {
  // telkens het window van grootte verandert,
  // wordt deze method opgeroepen
  //
  // Merk op dat de <div> voor layout redenen "leeg" is,
  // omdat het enige kind absoluut gepositioneerd is. Als
  // we niets speciaals doen zal het dus 0px hoog zijn.
  // Daarom stellen we hier programmatorisch de breedte
  // en hoogte in, zodat het altijd alle ruimte inneemt.
  let speelveld=document.getElementById("speelveld");
  speelveld.style.width=window.innerWidth+"px";
  speelveld.style.height=window.innerHeight+"px";
}

const moveSprite = () => {
  // Deze function wordt opgeroepen telkens de gebruiker
  // op het speelveld klikt.

  let sprite=document.getElementsByClassName("sprite")[0];
  let speelveld=document.getElementById("speelveld");
  let maxLeft=speelveld.clientWidth - sprite.offsetWidth;
  let maxHeight=speelveld.clientHeight - sprite.offsetHeight;

  // verplaats de sprite
  let left=Math.floor(Math.random()*maxLeft);
  let top=Math.floor(Math.random()*maxHeight);
  sprite.style.left=left+"px";
  sprite.style.top=top+"px";
}

window.addEventListener("load", setup);
```

DEMO sprites animatie



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <title>Demo</title>
  <link rel="stylesheet" href="../styles/demo_sprites_animatie.css">
</head>

<body>
  <div id="speelveld" class="unselectable">
    
  </div>
  <Script src="../scripts/demo_sprites_animatie.js"></script>
</body>
```

Javascript

```
let UPDATE_DELAY=50; // constante voor timer delay, in milliseconden
let speedX;
let speedY;

const setup = () => {
  window.addEventListener("resize", updateSize);

  // al 1 keer op voorhand oproepen, voor het geval
  // dat de gebruiker nooit manueel het browservenster
```



```

// groter of kleiner maakt
updateSize();

let speelveld=document.getElementById("speelveld");
let sprite=document.getElementsByClassName("sprite")[0];
sprite.style.left=parseInt(speelveld.style.width)/2+"px";
sprite.style.top=parseInt(speelveld.style.height)/2+"px";

// start een timer die om de 50ms de moveSprite method oproept
setInterval(moveSprite, UPDATE_DELAY);
}

const updateSize = () => {
  // telkens het window van grootte verandert,
  // wordt deze method opgeroepen
  //
  // Merk op dat de <div> voor layout redenen "leeg" is,
  // omdat het enige kind absoluut gepositioneerd is. Als
  // we niets speciaals doen zal het dus 0px hoog zijn.
  // Daarom stellen we hier programmatorisch de breedte
  // en hoogte in, zodat het altijd alle ruimte inneemt.
  let speelveld=document.getElementById("speelveld");
  speelveld.style.width=window.innerWidth+"px";
  speelveld.style.height=window.innerHeight+"px";

  speedX=window.innerWidth/(1000/UPDATE_DELAY); // 1x per seconde ganse
  breedte afleggen
  speedY=window.innerHeight/(1000/UPDATE_DELAY); // 1x per seconde ganse
  hoogte afleggen
}

const moveSprite = () => {
  // Deze function wordt ongeveer om de 50ms opgeroepen
  let sprite=document.getElementsByClassName("sprite")[0];
  let speelveld=document.getElementById("speelveld");

  let maxLeft=speelveld.clientWidth - sprite.offsetWidth;
  let maxTop=speelveld.clientHeight - sprite.offsetHeight;

  // bepaal de nieuwe positie van de sprite
  let left=parseInt(sprite.style.left); // begin met huidige left waarde
  left+=speedX; // pas aan voor horizontale snelheid
  let top=parseInt(sprite.style.top); // begin met huidige top waarde
  top+=speedY; // pas aan voor verticale snelheid

  // controleer of rand bereikt werd (horizontaal)
  if (left<0) {
    // linkerrand
    speedX=-speedX;
    left=0;
  } else if (left>maxLeft) {
    // rechterrand
    speedX=-speedX;
    left=maxLeft;
  }

  // controleer of rand bereikt werd (verticaal)
  if (top<0) {
    // bovenrand
    speedY=-speedY;
    top=0;
  }

```

```

    } else if (top>maxTop) {
        // onderrand
        speedY=-speedY;
        top=maxTop;
    }

    // verplaats de sprite
    sprite.style.left=left+"px";
    sprite.style.top=top+"px";
}

window.addEventListener("load", setup);

```

CSS

```

html, body {
    margin:0px;
}

#speelveld {
    position:relative;
    border:solid 1px black;
    box-sizing:border-box; /* anders krijgen we scrollbars omdat het
element door de border net te groot wordt */
}

.sprite {
    position:absolute;
}

.unselectable {
    -webkit-user-select: none;
    -moz-user-select: none;
    user-select: none;
}

```

hit an object

Hit an Object

Start spel

Score: 0



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="../styles/hit_an_object.css">
  <title>Hit an object</title>
</head>

<body>
<h1>Hit an Object</h1>

<button id="btnStart">Start spel</button>
<p>Score: <span id="score">0</span></p>

<div id="playfield">
  
</div>

<Script src="../scripts/hit_an_object.js"></script>
</body>
```

CSS

```
#playfield{
  width: 600px;
  height: 800px;
  border: 2px solid black;
  position: relative;
  margin-top: 20px;
}

#target{
  position: absolute;
  width: 48px;
  height: 48px;
  cursor: pointer;
}
```

Javascript

```
let global = {
  IMAGE_COUNT: 5, // aantal figuren
  IMAGE_SIZE: 48, // grootte van de figuur
  IMAGE_PATH_PREFIX: "../images/hit_an_object/", // map van de figuren
  IMAGE_PATH_SUFFIX: ".png", // extensie van de figuren
  MOVE_DELAY: 1000, // duurt 1 seconde totdat volgende afbeelding
  score: 0, // aantal punten
  timeoutId: 0 // id van de timeout timer, zodat we kunnen annuleren
}

let timerId=0;

const setup = () =>{
  let image = document.getElementById('target');
  image.addEventListener('click', geklikt);
  image.style.display = "none";
  document.getElementById("btnStart").addEventListener("click",
startGame);

  timerId=setInterval(consoleMelding, global.MOVE_DELAY);
}

const startGame = () => {
  // score resetten
  global.score = 0;
  document.getElementById("score").innerText = global.score;

  // sprite tonen
  let image = document.getElementById("target");
  image.style.display = "block";

  // timers starten
  timerId = setInterval(consoleMelding, global.MOVE_DELAY);
  moveImage();
};

const consoleMelding = () => {
  console.log("1 seconde gepasseerd")
}

const moveImage = () => {

  let image=document.getElementById("target");
  let playfield=document.getElementById("playfield");
  let maxLeft=playfield.clientWidth - image.offsetWidth;
  let maxHeight=playfield.clientHeight - image.offsetHeight;

  let left=Math.floor(Math.random()*maxLeft);
  let top=Math.floor(Math.random()*maxHeight);

  image.style.left=left+"px";
  image.style.top=top+"px";

  let randomImage = Math.floor(Math.random() * global.IMAGE_COUNT);
  image.src = global.IMAGE_PATH_PREFIX + randomImage +
global.IMAGE_PATH_SUFFIX;

  global.timeoutId = setTimeout(moveImage, global.MOVE_DELAY);
}
```

```
}

const geklikt = () => {
  let image = document.getElementById("target");

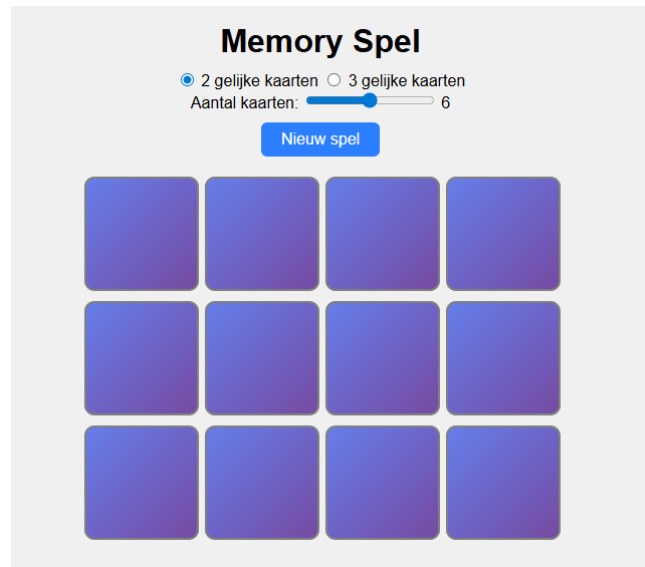
  if (image.src.includes("0.png")) {      // controleren als bom
    alert("GAME OVER");

    clearTimeout(global.timeoutId);
    clearInterval(timerId);

    image.style.display = "none";
    return;
  }
  global.score++;
  document.getElementById("score").innerText = global.score;
  clearTimeout(global.timeoutId);
  moveImage();
};

window.addEventListener("load", setup);
```

matching game



HTML

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <title>Memory Spel</title>
  <link rel="stylesheet" href="../styles/matching_game.css">
</head>
<body>

<h1>Memory Spel</h1>

<div id="opties">
  <label>
    <input type="radio" name="aantal_kaarten" value="2" checked>
    2 gelijke kaarten
  </label>
  <label>
    <input type="radio" name="aantal_kaarten" value="3">
    3 gelijke kaarten
  </label>
</div>

<div id="slider-opties">
  <label for="kaarten-slider">Aantal kaarten: </label>
  <input type="range" id="kaarten-slider" min="2" max="10" value="6"
step="1">
  <span id="kaarten-waarde">6</span>
</div>

<button id="nieuw_game_btn">Nieuw spel</button>
<div id="speelveld"></div>
<div id="bericht"></div>

<script src="../scripts/matching_game.js"></script>
</body>
</html>
```

CSS

```
body {
  font-family: Arial, sans-serif;
  background-color: #f0f0f0;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 20px;
}

h1 {
  margin-bottom: 10px;
}

#speelveld {
  display: grid;
  grid-template-columns: repeat(4, 110px);
  gap: 10px;
  margin-top: 20px;
}

.kaart {
  width: 110px;
  height: 110px;
  cursor: pointer;
  border-radius: 10px;
  border: 2px solid gray;
  /* Achterkant = CSS gradient, standaard zichtbaar */
  background: linear-gradient(135deg, #667eea, #764ba2);
}

/* Omgedraaid: verberg gradient, toon de afbeelding */
.kaart.omgedraaid {
  background: none;
}

.kaart img.voorkant { display: none; }
.kaart.omgedraaid img.voorkant { display: block; width: 100%; height: 100%; object-fit: cover; border-radius: 8px; }

/* Gevonden: behoud ruimte maar maak onzichtbaar */
.kaart.gevonden {
  visibility: hidden;
}

/* Feedback: goed of fout */
.kaart.goed {
  border-color: green;
  box-shadow: 0 0 8px green;
}

.kaart.fout {
  border-color: red;
  box-shadow: 0 0 8px red;
}

/* Wachtijd: draaiende laad cursor */
body.bezet { cursor: wait; }
body.bezet .kaart { cursor: wait; }
```

```

#bericht {
  margin-top: 20px;
  font-size: 1.4em;
  font-weight: bold;
  color: green;
  min-height: 2em;
}

button {
  margin-top: 10px;
  padding: 8px 20px;
  font-size: 1em;
  cursor: pointer;
  border-radius: 6px;
  border: none;
  background-color: #2b7fff;
  color: white;
}

button:hover {
  background-color: #1a5fcc;
}

```

JavaScript

```

let global = {
  // — Globale constanten —————
  AANTAL_KAARTEN: 6,           // standaard 6 paren
  AANTAL_GELIJKE_KAARTEN: 2,   // standaard 2 kaarten gelijk te stellen

  AFBEELDINGEN: [             // lijst met paden naar de afbeeldingen
    '../images/matching_game/kaart1.png',
    '../images/matching_game/kaart2.png',
    '../images/matching_game/kaart3.png',
    '../images/matching_game/kaart4.png',
    '../images/matching_game/kaart5.png',
    '../images/matching_game/kaart6.png',
    '../images/matching_game/kaart7.png',
    '../images/matching_game/kaart8.png',
    '../images/matching_game/kaart9.png',
    '../images/matching_game/kaart10.png',
  ],

  GELUID_GOED: new Audio('../sounds/sound_good.mp3'), // geluid bij juist antwoord
  GELUID_FOUT: new Audio('../sounds/sound_bad.mp3'),  // geluid bij fout antwoord
};

let omgedraaid = []; // lijst van kaarten die momenteel omgedraaid zijn
let isBezig    = false; // true als spel even wacht
let gevonden   = 0; // aantal gevonden paren/trio

// — Setup speelveld —————
const setup = () => {
  gevonden = 0; //standaard gevonden bij start van spel = 0

  // beide radiobuttons overlopen om te weten of het duo's of trio's zijn die gezocht worden
  const radios = document.getElementsByName("aantal_kaarten");
  for (let i = 0; i < radios.length; i++) {
    if (radios[i].checked) {
      global.AANTAL_GELIJKE_KAARTEN = parseInt(radios[i].value);
    }
  }

  // Lees de slider uit om te weten hoeveel combinaties
  global.AANTAL_KAARTEN = parseInt(document.getElementById("kaarten-slider").value);
  // toon sliderwaarde naast de slider
  document.getElementById("kaarten-waarde").textContent = global.AANTAL_KAARTEN;
}

```



```

    const totaal = global.AANTAL_KAARTEN * global.AANTAL_GELIJKE_KAARTEN; // totaal aantal kaarten in spel
    const kolommen = berekenKolommen(totaal); // bereken beste aantal kolommen voor bovenstaand totaal

    const speelveld = document.getElementById("speelveld"); // speelveld ophalen
    speelveld.innerHTML = ""; // alle oude kaarten verwijderen
    speelveld.style.gridTemplateColumns = "repeat(" + kolommen + ", 110px)"; // grid instellen met berekend aantal kolommen

    let kaarten = []; // verzameling van alle kaarten
    for (let i = 0; i < global.AANTAL_KAARTEN; i++) { // loop over elke kaart
        for (let j = 0; j < global.AANTAL_GELIJKE_KAARTEN; j++) { // voeg alle kaarten x keer toe
            kaarten.push(i); // kaartnummer in verzameling steken
        }
    }
    kaarten = shuffle(kaarten); // kaarten shufflen om op random plaats te zetten
    for (let i = 0; i < kaarten.length; i++) {
        speelveld.appendChild(maakKaart(kaarten[i])); // maak een kaart en voeg toe aan speelveld
    }
    document.getElementById("bericht").textContent = ""; // verwijder winnaarsbericht
};

// shuffle methode om array willekeurig te sorteren
const shuffle = (array) =>
    array.sort(() => Math.random() - 0.5);

// methode om beste kolom te berekenen
const berekenKolommen = (totaal) => {
    const verhouding = window.innerWidth / window.innerHeight; // verhouding scherm (breedte/hoogte)

    let besteKolommen = totaal; // beginwaarde: alles op één rij
    let kleinsteVerschil = Infinity; // beginwaarde: oneindig groot verschil

    for (let kolommen = 1; kolommen <= totaal; kolommen++) { // probeer elk mogelijk aantal kolommen
        if (totaal % kolommen !== 0) continue; // als er overblijvers zijn --> overslaan

        const rijen = totaal / kolommen; // bereken aantal rijen
        const gridVerhouding = kolommen / rijen; // verhouding van grid
        const verschil = Math.abs(gridVerhouding - verhouding); // verschil met schermverhouding

        if (verschil < kleinsteVerschil) { // is dit beste verdeling tot nu toe?
            kleinsteVerschil = verschil; // onthoud kleinste verschil
            besteKolommen = kolommen; // onthoud bijhorend aantal kolommen
        }
    }
    return besteKolommen; // geef beste aantal kolommen weer
};

// kaarten aanmaken
const maakKaart = (index) => {
    const kaart = document.createElement("div"); // maak een div aan voor de kaarten
    kaart.className = "kaart" // geef de div klasse "kaart"
    const img = document.createElement("img"); // maak een img voor de afbeeldingen
    img.src = global.AFBEELDINGEN[index]; // afbeelding instellen op basis van index
    img.className = "voorkant"; // geef klasse "voorkant"
    kaart.appendChild(img); // voeg afbeelding toe aan de kaart
    kaart.addEventListener("click", () => klikKaart(kaart, index)); // voeg een klik-listener toe
    return kaart; // geef kaart terug
};

// Wanneer op de kaart geklikt wordt
const klikKaart = (kaart, index) => {
    if (isBezig) return; // doe niets als het spel bezig is
    if (kaart.classList.contains("omgedraaid")) return; // doe niets als kaart al omgedraaid is
    if (kaart.classList.contains("gevonden")) return; // doe niets als kaart al gevonden is
    kaart.classList.add("omgedraaid"); // draai kaart om
    omgedraaid.push({ kaart: kaart, index: index }); // voeg kaart toe aan omgedraaide lijst
    if (omgedraaid.length === global.AANTAL_GELIJKE_KAARTEN) { // als er genoeg kaarten

```

```

omgedraaid zijn
    controleerMatch(); // controleren op match
}
};

// controleren als er een match is
const controleerMatch = () => {
    isBezig = true; // zorg dat er niet geklikt kan worden
    document.body.classList.add("bezet"); // zet cursor op laden

    const eersteIndex = omgedraaid[0].index; // onthoud index van eerste kaart
    let gelijk = true; // veronderstel alle kaarten gelijk zijn
    for (let i = 1; i < omgedraaid.length; i++) { // loop over andere omgedraaide kaarten
        if (omgedraaid[i].index !== eersteIndex) { // als kaart anders is dan eerste
            gelijk = false; // niet gelijk
        }
    }

    if (gelijk) { // als gelijk
        for (let i = 0; i < omgedraaid.length; i++) {
            omgedraaid[i].kaart.classList.add("goed"); // toon groene rand
        }
        global.GELUID_GOED.currentTime = 0; // geluid start vanaf begin
        global.GELUID_GOED.play(); // geluid speelt af
        setTimeout(() => { // wacht voor kaarten terugdraaien
            for (let i = 0; i < omgedraaid.length; i++) {
                omgedraaid[i].kaart.classList.add("gevonden"); // kaarten verbergen
            }
            gevonden++; // aantal gevonden omhoog met 1
            resetBeurt(); // reset de beurt
            if (gevonden === global.AANTAL_KAARTEN) { // als alle paren gevonden zijn
                // toon winnaarsbericht
                document.getElementById("bericht").textContent = "Gefeliciteerd! Je hebt
gewonnen!";
            }
        }, 500); // wacht halve seconde
    } else {
        for (let i = 0; i < omgedraaid.length; i++) {
            omgedraaid[i].kaart.classList.add("fout"); // toon rode rand rond de gekozen
kaarten
        }
        global.GELUID_FOUT.currentTime = 0; // geluid start vanaf begin
        global.GELUID_FOUT.play(); // geluid speelt af
        setTimeout(() => {
            for (let i = 0; i < omgedraaid.length; i++) {
                omgedraaid[i].kaart.classList.remove("omgedraaid"); // draai kaart terug om
                omgedraaid[i].kaart.classList.remove("fout"); // verwijder rode rand weer
            }
            resetBeurt(); // reset de beurt
        }, 1000); // wacht seconde
    }
};

// beurt resette,
const resetBeurt = () => {
    omgedraaid = []; // maak lijst met omgedraaide kaarten leeg
    isBezig = false; // je kan weer kaarten kiezen
    document.body.classList.remove("bezet"); // zet cursor normaal
};

// — Radiobuttons starten setup direct —
const radios = document.getElementsByName("aantal_kaarten"); // haal alle radiobuttons op
for (let i = 0; i < radios.length; i++) {
    radios[i].addEventListener("change", setup); // start setup opnieuw bij elke wijziging
}

// — Slider past aantal kaarten live aan —
document.getElementById("kaarten-slider").addEventListener("input", setup); // start setup
opnieuw bij sliden

// spel starten
document.getElementById("nieuw_game_btn").addEventListener("click", setup); // klik op knop
start nieuw spel
window.addEventListener("load", setup); // start spel automatisch bij laden

```